

Course Introduction

Welcome to Python Programming

- ✔ **Video:** Introduction to Programming
4 min
- ✔ **Video:** Why Python?
2 min
- ✔ **Reading:** Visual Studio Code
5 min
- ✔ **Reading:** Installing Python paths (Optional for Windows Users)
5 min
- ✔ **Reading:** Installing Python paths (Optional for Mac users)
5 min
- ✔ **Reading:** Required dependencies
5 min
- ✔ **Video:** Environment check for Windows
2 min
- 🔒 **Video:** Environment check for Mac
3 min
- ✔ **Video:** Running code - Command line VS IDE
3 min
- ✔ **Video:** Python syntax, spaces matter
3 min
- ✔ **Reading:** Python syntax cheat sheet
- ✔ **Reading:** Commenting code
5 min
- ✔ **Video:** Variables
6 min
- ✔ **Video:** Basic data types
6 min
- ✔ **Video:** Strings
5 min
- ✔ **Reading:** Basic Data type and Function Cheatsheet
10 min
- ✔ **Video:** Type casting
2 min
- ✔ **Video:** User input, console output
8 min
- ✔ **Reading:** Type casting, a deeper look
10 min
- 🔒 **Programming Assignment:** Type casting input
3h
- 🔒 **Practice Quiz:** Knowledge check - Welcome to Python Programming
5 questions
- 🔒 **Reading:** Additional resources
5 min

Control flow and conditionals

Type casting, a deeper look

There are some scenarios in which a given value's data type needs to be changed to some other data type.

This process is known as type casting.

Another, more informal way to refer to it is "data type conversion".

The simplest example of converting data could be the following comparison:

```
1 print(10 == 10)
```

In the above piece of code, I'm asking Python if the number 10 is equal to the number 10, and I'll get the boolean value of **True** printed, confirming that indeed, they are the same.

What if I do this?

```
1 print(10 == 10.00)
```

Again, the boolean value of **True** is output on the screen.

Now, you might complain that, well, 10 is not *technically* equal to 10.0 - because, one might argue, the first number is an integer, and the second number is a float. You would be right - although these are the same numbers, they are not the same data types.

However, Python here performs what's known as "**implicit type conversion**".

To understand how this works, I'll slightly tweak the previous example. Instead of asking Python to compare the two numbers and return if they are the same or not, I'll ask Python to print the result of adding these two numbers together.

```
1 print(10 + 10.0)
```

The printed result is 20.0.

This output allows me to conclude that **when Python runs operations involving integers and floats, it implicitly converts the integers type to a float, and then completes the operation.**

To really drive this point home, I can extend my previous example, using the **type()** function:

```
1 print(type(10 + 10.0))
```

The output is `<class 'float'>`, so this confirms my conclusion.

Let me know show you a small program in Python, working with numbers:

```
1 user_num_1 = input('First number is: ')
2 user_num_2 = input('Second number is: ')
3 user_sum = user_num_1 + user_num_2
4 print(user_sum)
```

When I run this program, I could, for example, provide the first number's value as 5, and the second number's value also as 5, expecting that the printed **user_sum** value will be 10.

However, when I do exactly that, the number **55** gets printed instead.

Why was this not working?

Well, the answer is pretty simple: everything that a user types in, is converted, by Python, to the string data type.

This means that, although I typed numbers into these two inputs, what was saved in **user_num_1** and **user_num_2** were actually strings.

Effectively, it's exactly the same as if I just did this:

```
1 user_num_1 = "5"
2 user_num_2 = "5"
3 user_sum = user_num_1 + user_num_2
4 print(user_sum)
```

This time, the output of printing **user_sum** is still "55", but this comes at no surprise.

In order to have my Python code work as I intended, I need to perform **explicit type conversion**.

In other words, I have to convert the value of "5" to the value of 5.

Here's my updated code:

```
1 user_num_1 = input('First number is: ')
2 user_num_2 = input('Second number is: ')
3 user_sum = float(user_num_1) + float(user_num_2)
4 print(user_sum)
```

What I'm doing here is, I'm making sure that my program can handle even accepting floats as inputs, and still work correctly.

In other words, I'm making sure that if a user provided, say, the float value of 5.5 as both the first and second numbers when running the above code, the output would not throw an error. Instead, it will be the expected 11.0.

What if I decide to output some words to the user, telling them what happened?

Here's an attempt at doing that:

```
1 num_1 = input('First number is: ')
2 num_2 = input('Second number is: ')
3 user_sum = float(num_1) + float(num_2)
4 print("The sum of: " + num_1 + " and " + num_2 + " is " + user_sum)
```

If I run the above code, it will throw the following error:

Traceback (most recent call last):
File "<string>", line 4, in <module>
TypeError: can only concatenate str (not "float") to str

What this means is, I cannot concatenate a string and a float like that. In other words, although Python's **implicit type conversion** works when I use the + operator *on strings and integers*, it does *not work on strings and floats*.

The solution to this is to perform explicit type conversion, as follows.

```
1 n1 = input('First number is: ')
2 n2 = input('Second number is: ')
3 user_sum = float(n1) + float(n2)
4 print("The sum of " + str(n1) + " and " + str(n2) + " is " + str(user_sum))
```

This time, the output is: **The sum of 5.5 and 5.5 is 11.0.**

In Python, it's easy to perform explicit conversions, and sometimes they are very useful. You'll learn more about how this works as you get more experience in Python.

✔ **Completed** **Go to next item**

👍 Like 🗨 Dislike 📄 Report an issue